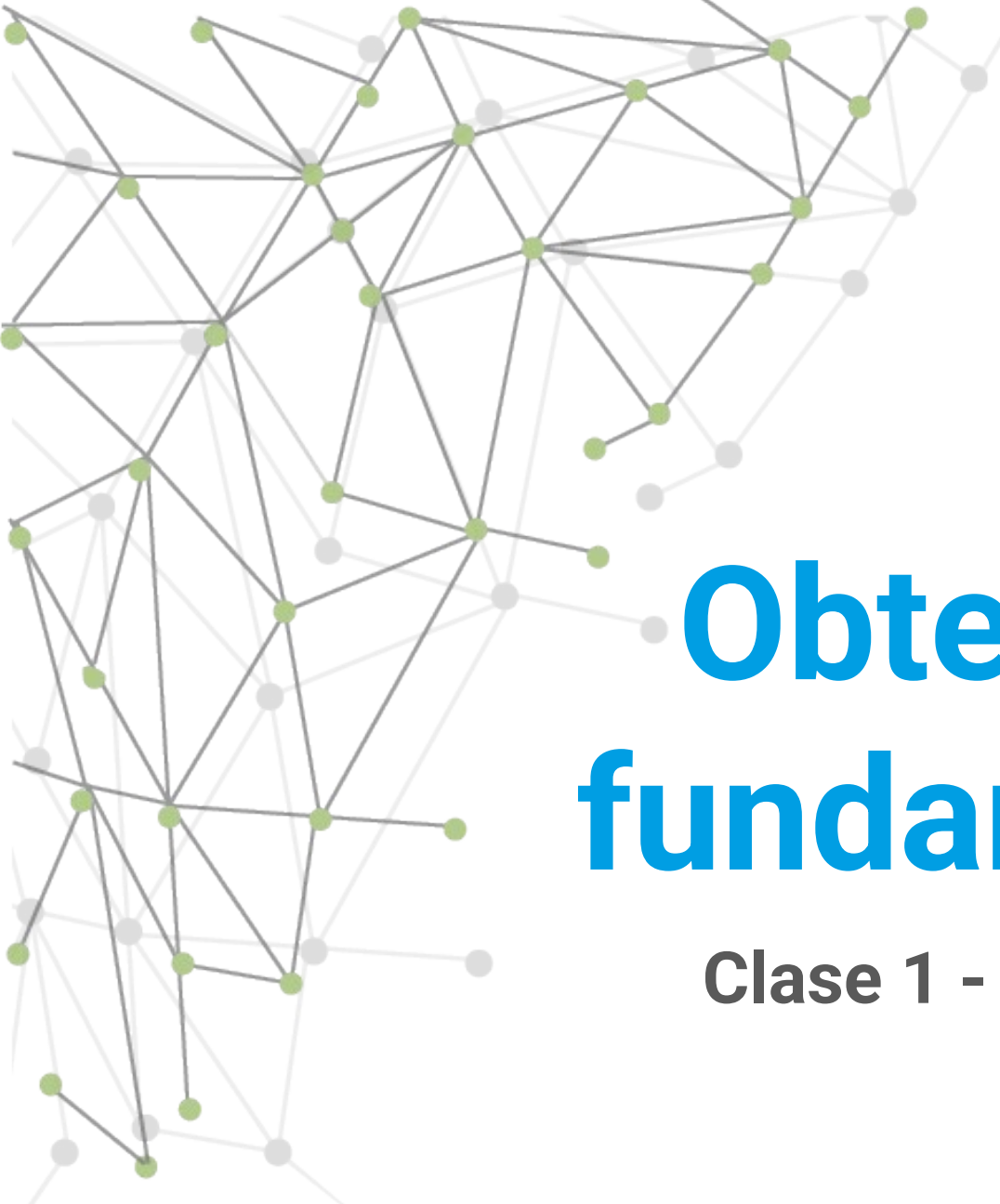
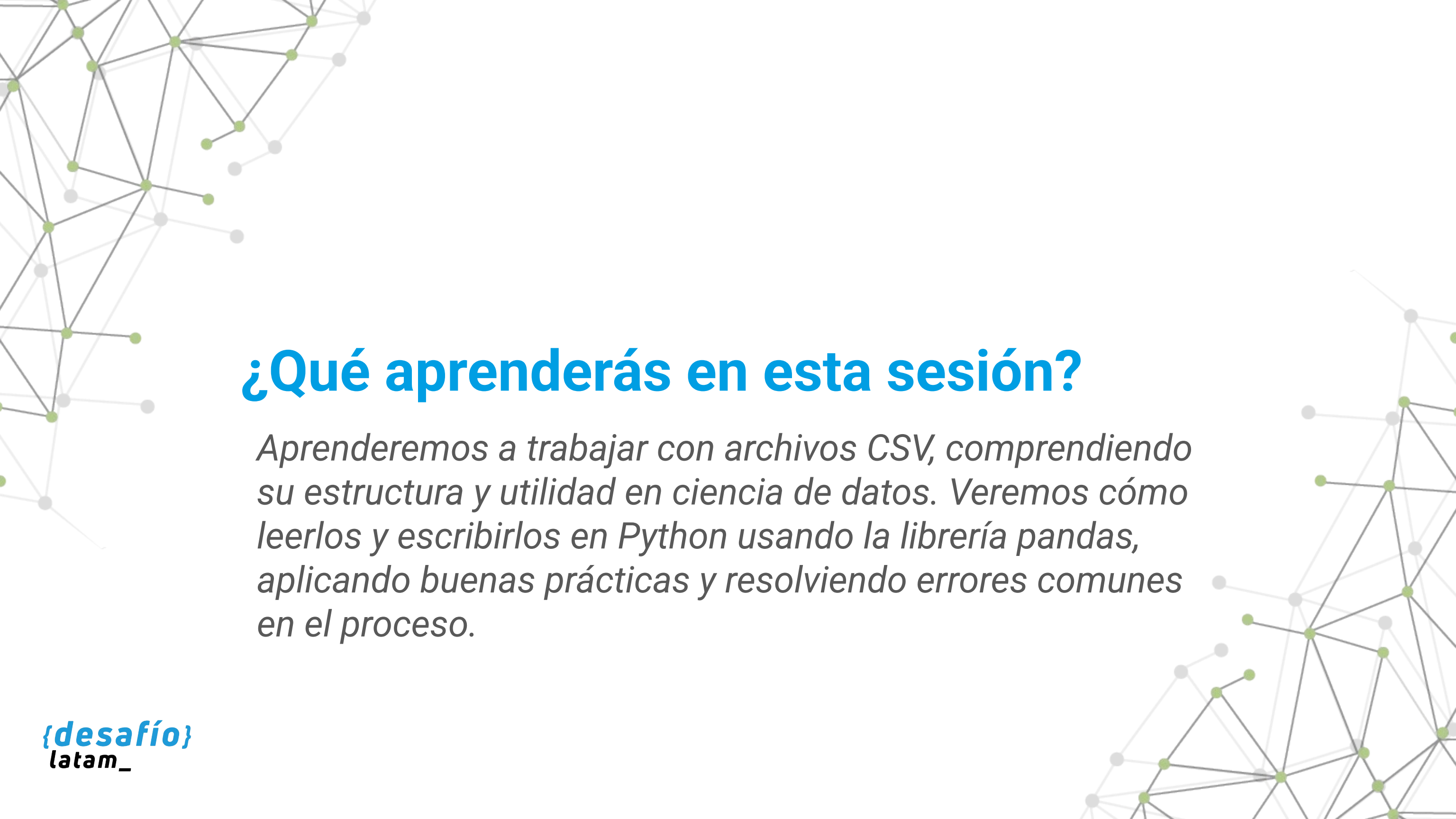




Obtención de datos y fundamentos de Numpy

Clase 1 - Lectura y escritura de archivos CSV





¿Qué aprenderás en esta sesión?

Aprenderemos a trabajar con archivos CSV, comprendiendo su estructura y utilidad en ciencia de datos. Veremos cómo leerlos y escribirlos en Python usando la librería pandas, aplicando buenas prácticas y resolviendo errores comunes en el proceso.



Desafío inicial

Trabajas en una empresa de distribución de alimentos. Cada sucursal te envía diariamente un archivo CSV con el registro de ventas, que luego debes consolidar y analizar. Tu jefatura te pide automatizar este proceso, pero primero necesitas dominar cómo se leen, interpretan y generan correctamente estos archivos en Python.

¿Cómo podrías asegurar una lectura y escritura precisa y útil de estos archivos?

Para resolver esta necesidad, deberás conocer cómo leer y escribir archivos CSV desde Python utilizando la biblioteca pandas, controlando aspectos como separadores, codificación, nombres de columnas y validación de datos para asegurar un procesamiento preciso y automatizable.

Lectura de archivos

Dominar la función `read_csv()` y sus parámetros

Procesamiento de datos

Transformar y analizar la información con pandas

Escritura de archivos

Exportar resultados con `to_csv()` aplicando buenas prácticas

**/* Obtención de datos desde archivos
CSV */**

¿Qué es un archivo CSV?

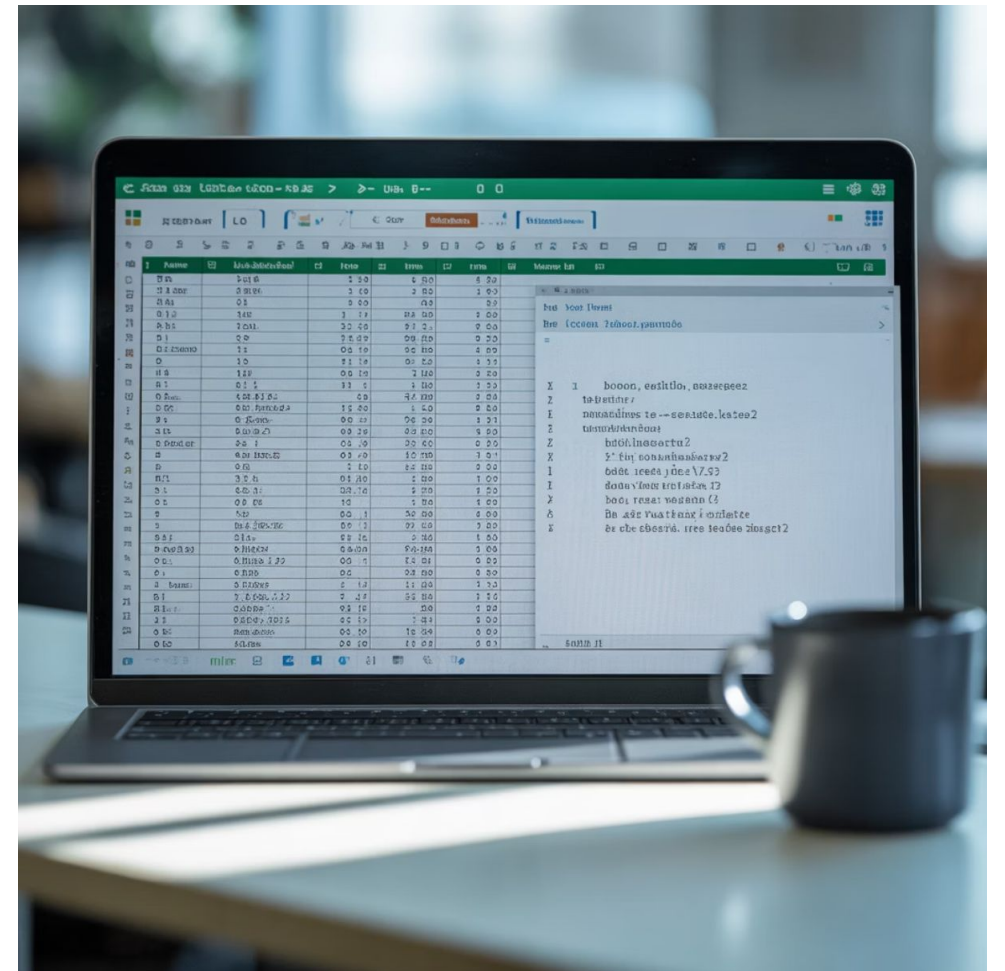
Un archivo CSV (Comma Separated Values) es un archivo de texto plano en el que los datos están organizados en forma de tabla. Cada línea representa una fila de datos y cada valor dentro de la línea corresponde a una columna. Los valores se separan mediante un delimitador, que por defecto suele ser la coma (,), aunque puede variar.

[illegible]

Ejemplo simple de archivo CSV

```
producto,cantidad,precioPan,10,1200Leche,5,1500
```

Al abrirlo con Excel, se verá como una tabla. Al abrirlo con un bloc de notas, se verán las comas como separadores. Es el formato preferido para exportar e intercambiar datos entre sistemas y plataformas.



¿Por qué el CSV es tan utilizado?

El CSV es uno de los formatos más populares para intercambio de datos entre sistemas. Sus principales ventajas son:

-  Es ligero, ya que no contiene estilos, fórmulas ni macros.
-  Es interoperable: puede ser leído por programas como Excel, Google Sheets, bases de datos o software de análisis como Python y R.
-  Es transparente: se puede abrir y revisar con cualquier editor de texto.
-  Es ideal para automatización, porque se puede generar o leer fácilmente desde cualquier lenguaje de programación.

Ejemplo: El portal datos.gob.cl entrega series estadísticas públicas en CSV para facilitar su uso en distintos programas.

¿Cómo está estructurado internamente un archivo CSV?

Un archivo CSV tiene tres componentes clave:

1

Cabecera (opcional pero recomendada): Primera fila con nombres de columnas. Ejemplo:
`rut;nombre;ventas`

2

Registros: Cada fila representa un caso, observación o entidad.

3

Delimitador: El separador de los campos puede variar:

- , (coma) – estándar en EE.UU.
- ; (punto y coma) – común en Europa y Chile.
- \t (tabulador) – archivos TSV.
- | (barra vertical) – en sistemas con campos textuales con comas.

Ejemplo con delimitador (punto y coma)

```
rut;nombre;ventas11111111-1;Pedro  
Soto;4500022222222-2;Juana Reyes;62000
```

Importante: Si un campo contiene comas, debe estar entre comillas dobles:

```
nombre,direccion"Ana Torres","Calle 123,  
Depto 5B"
```



Consideraciones al usar archivos CSV

Aunque el CSV es muy útil, también tiene limitaciones:



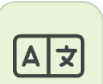
Sin formato visual

No permite formatos visuales (colores, negritas).



Sin validación

No tiene validación de tipos o fórmulas.



Problemas de codificación

Puede perder información si no se codifica correctamente (ej. tildes mal leídas si no se usa UTF-8).



Estructura simple

No admite estructuras complejas (tablas anidadas, jerarquías).

Por ello, es importante revisar y validar bien el contenido antes de usarlo.

/* Leyendo un archivo CSV */



¿Qué es pandas y por qué es esencial en ciencia de datos?

pandas es una biblioteca de Python diseñada para facilitar la manipulación y análisis de datos estructurados. Su elemento central es el DataFrame, una estructura de datos bidimensional similar a una hoja de cálculo, donde los datos se organizan en filas y columnas.

Importando pandas

Importar la biblioteca es el primer paso:

```
import pandas as pd
```

El rol de pandas en el flujo de trabajo de ciencia de datos es fundamental: permite cargar, transformar, limpiar, analizar y exportar datos de forma eficiente y expresiva.



Lectura básica con `read_csv()`

Supongamos que tienes un archivo llamado `ventas.csv` con este contenido:

```
producto,cantidad,precioPan,10,1200Leche,5,1500
```


Código para cargar el archivo CSV

```
df = pd.read_csv('ventas.csv')  
print(df)
```



Resultado en yaml

```
producto cantidad precio0 Pan 10 12001 Leche
5 1500
```



Observaciones



Índices automáticos

pandas crea automáticamente una columna de índices numéricos (0, 1, 2, ...).



Cabecera como columnas

La cabecera del CSV se transforma en los nombres de las columnas del DataFrame.



Filas de datos

Cada fila del archivo pasa a ser una fila de datos.

Parámetros clave en read_csv()

Aunque el uso básico es muy directo, read_csv() permite personalizar el proceso mediante parámetros opcionales, lo que es vital al tratar con archivos complejos o mal estructurados.

Parámetro	Descripción	Ejemplo
sep	Define el separador de campos (coma por defecto)	sep=';'
encoding	Controla la codificación de caracteres (útil para tildes, ñ, etc.)	encoding='utf-8' o encoding='latin1'
header	Especifica si hay fila de cabecera	header=None si no hay
skiprows	Omite una o más filas al inicio del archivo	skiprows=2
usecols	Carga solo columnas específicas por nombre o posición	usecols=['producto', 'precio']

Estos parámetros permiten controlar qué datos se cargan, cómo se interpretan y cómo se transforman internamente en Python.

Leer un CSV desde internet (URL)

Una de las grandes ventajas de pandas es que puede leer archivos directamente desde una URL:

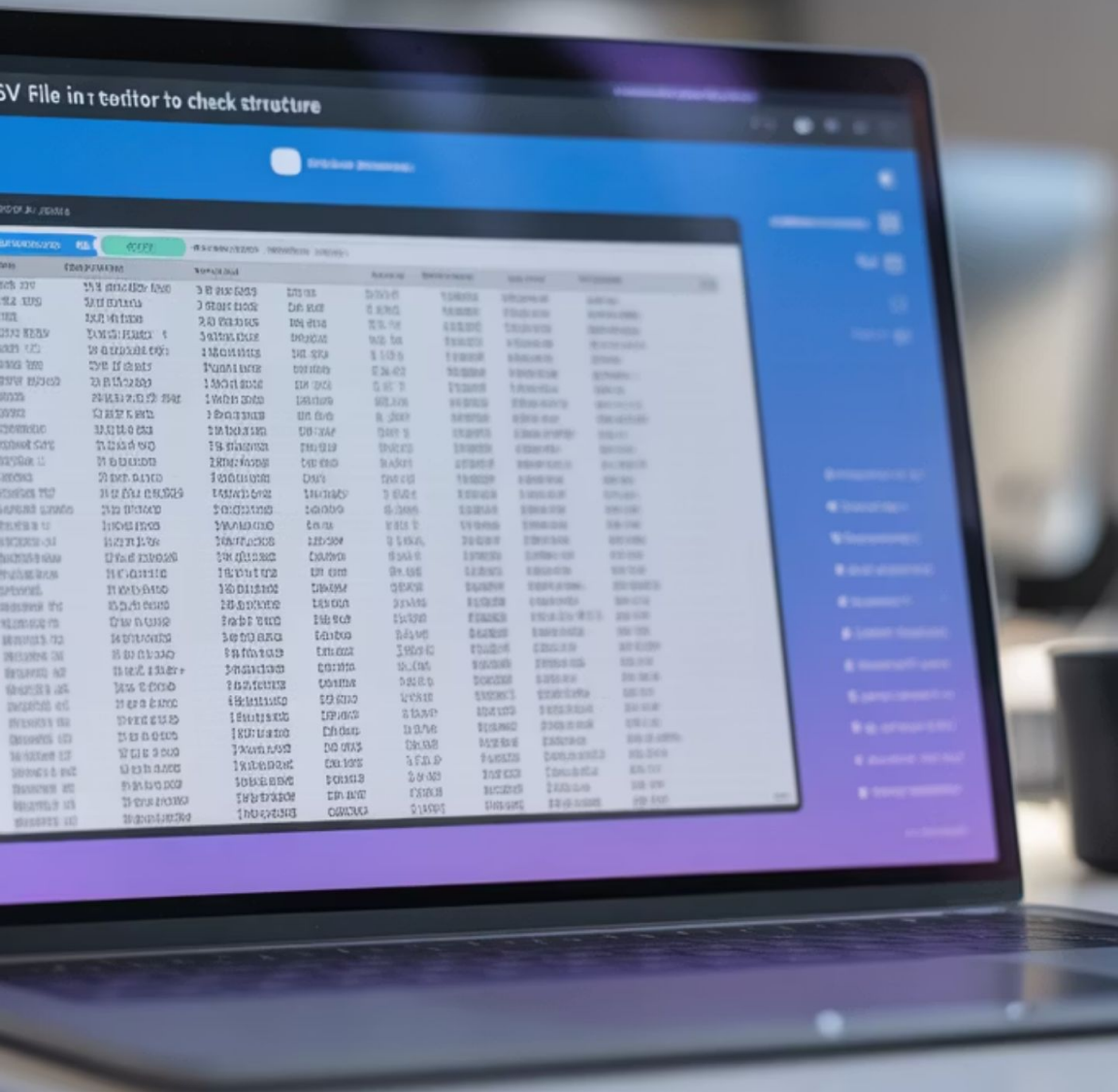
```
url = 'https://people.sc.fsu.edu/~jburkardt/data/csv/airtravel.csv'  
df = pd.read_csv(url)  
print(df.head())
```

Esto es ideal para trabajar con datos abiertos o actualizaciones automáticas desde fuentes oficiales o APIs públicas que entregan CSV.



Diagnóstico y solución de errores comunes

Problema	Posible causa	Solución
Todos los datos aparecen en una sola columna	Separador incorrecto	Usar <code>sep=';</code> , <code>sep='\t'</code> , etc.
Caracteres extraños (¿, ñ, tildes mal)	Codificación incorrecta	Probar <code>encoding='utf-8'</code> o <code>encoding='latin1'</code>
No se detecta la cabecera	Archivo sin fila de nombres	Usar <code>header=None</code> y asignar nombres con <code>df.columns = [...]</code>
Nombres de columnas mal interpretados	Fila desplazada o en blanco	Usar <code>skiprows</code> o revisar con <code>df.head()</code>



¡Importante!

Antes de intentar procesar el archivo, abre el CSV con un editor de texto para detectar visualmente el separador, codificación y cabeceras.

Verificación inicial del DataFrame

Luego de cargar el archivo, es buena práctica revisar su contenido con comandos de diagnóstico:

```
print(df.head()) # Primeras 5 filas  
print(df.info()) # Tipos de datos, cantidad de valores no nulos  
print(df.shape) # (filas, columnas)  
print(df.columns) # Lista de nombres de columnas
```

Esto te ayuda a:

- Validar que las columnas fueron interpretadas correctamente.
- Identificar tipos de datos (texto, números, fechas).
- Detectar errores tempranos.



Transición a escritura de archivos

Ahora que aprendimos cómo leer correctamente un archivo CSV desde distintas fuentes y cómo evitar errores comunes...¿cómo podemos guardar nuestros análisis y transformaciones en un nuevo archivo CSV para compartir con otras personas o integrarlos a otros sistemas?

/* Escribiendo un archivo CSV*/

¿Por qué exportar datos como CSV?

Después de procesar o transformar un conjunto de datos, generalmente queremos guardar los resultados para:



Respaldo

Usarlos como respaldo.



Compartir

Compartirlos con otros equipos.



Integración

Alimentar otros programas o sistemas
que requieren entradas estructuradas.



Documentación

Documentar resultados como evidencia
de un análisis.



Automatización

Automatizar reportes periódicos en
procesos organizacionales.

Ventajas del formato CSV para exportación

El formato CSV es ideal porque:



Ligereza

Es ligero y fácil de transferir.



Compatibilidad

Es universalmente compatible.



Versatilidad

Puede abrirse con Excel, Google Sheets, herramientas BI o editores de texto.

Escritura básica con to_csv()

La forma más simple de guardar un DataFrame como CSV:

```
df.to_csv('resultados.csv')
```

Esto genera un archivo .csv que incluye:

- Las columnas del DataFrame.
- Una columna de índice agregada automáticamente (0, 1, 2...).

¡Importante! Este índice puede generar errores si otro sistema espera un número exacto de columnas.

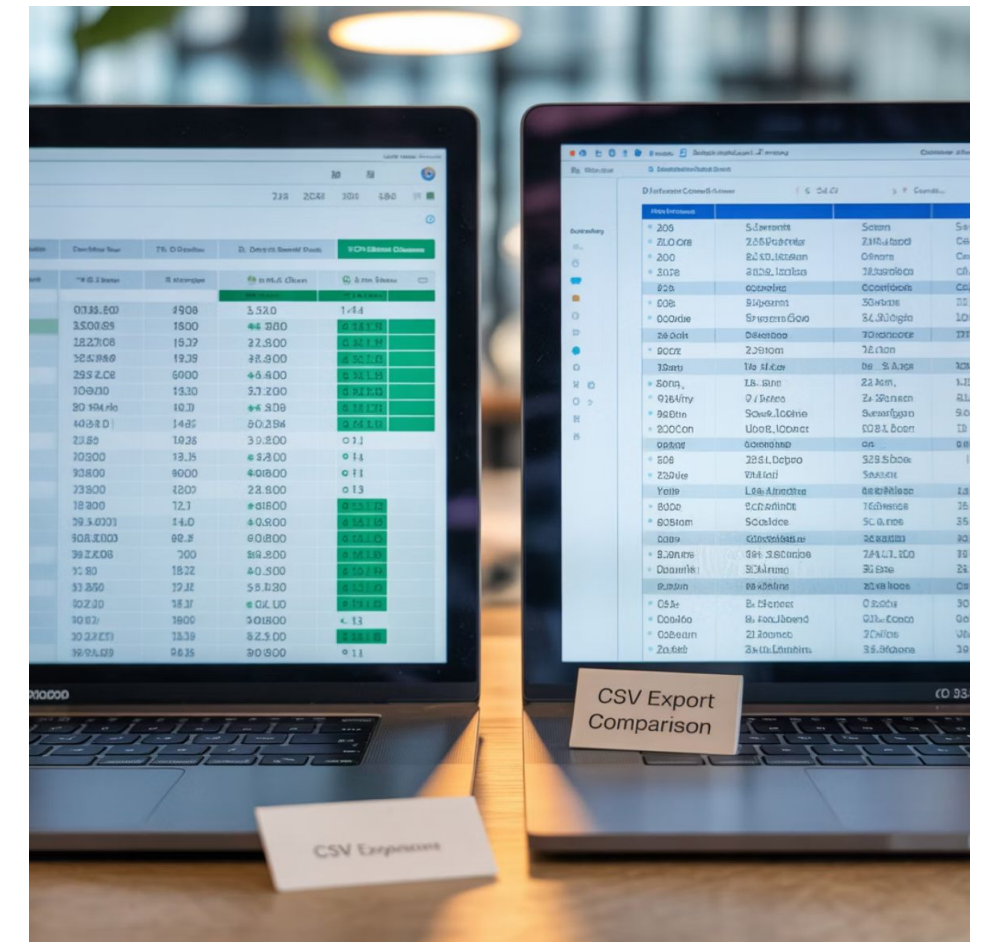
Exclusión del índice automático

Para evitar que pandas escriba una columna de índice (0, 1, 2...), se utiliza:

```
df.to_csv('resultados.csv', index=False)
```

Esto es recomendable si:

- El archivo será cargado en Excel, Power BI o sistemas contables.
- Se integrará con plataformas que validan estructura de columnas.



Cambiar el delimitador

En muchos países (como Chile, España o Francia) se usa la coma como separador decimal, por lo que es mejor utilizar ; como delimitador de columnas:

```
df.to_csv('datos.csv', sep=';', index=False)
```

¡Importante! Al compartir archivos con otras personas, especifica siempre el separador utilizado.

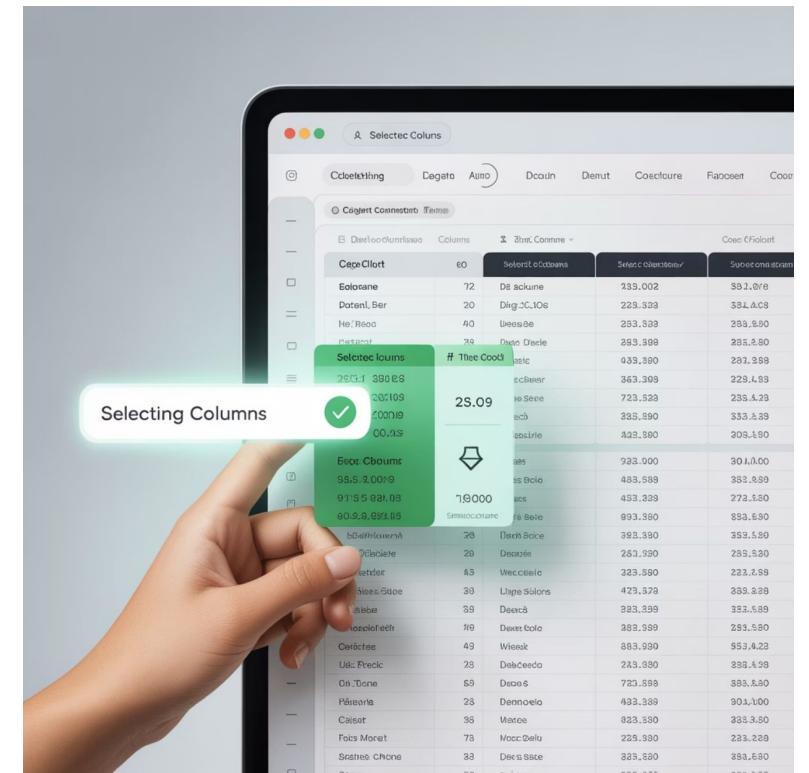
Seleccionar columnas específicas

Puedes guardar solo algunas columnas:

```
df[['producto', 'precio']].to_csv('productos.csv', index=False)
```

Esto permite:

- Proteger información sensible.
- Generar reportes resumidos.
- Optimizar el tamaño del archivo.



Controlar formato de números y fechas

Para garantizar consistencia en la salida de precios, porcentajes o fechas:

```
df.to_csv('resumen.csv', float_format='%.2f', date_format='%Y-%m-%d', index=False)
```



Formato de decimales

`float_format='%.2f'`: Fuerza dos decimales.



Formato de fechas

`date_format='%Y-%m-%d'`: Exporta fechas en formato ISO (recomendado).

Esto es especialmente importante cuando el CSV alimentará sistemas automatizados que requieren formato estándar.

Exportar archivos filtrados

Puedes aplicar un filtro antes de exportar:

```
filtrado = df[df['precio'] > 1000]  
filtrado.to_csv('filtrado.csv', index=False)
```

Esto permite:

- Automatizar reportes con condiciones específicas.
- Generar versiones segmentadas del mismo conjunto de datos.



Verificación del archivo generado

Después de exportar, es buena práctica validar el archivo:

```
verificacion = pd.read_csv('resultados.csv')  
print(verificacion.head())
```

Esto ayuda a:



Detectar errores

Detectar errores de codificación.



Confirmar formato

Confirmar formato y
delimitadores.



Validar contenido

Validar contenido antes de
enviarlo a otras personas o
sistemas.

Errores frecuentes al usar to_csv()

Error	Causa probable	Solución
Columna extra al abrir en Excel	No usaste index=False	Agrega index=False
Caracteres especiales mal interpretados	Codificación incorrecta	Usa encoding='utf-8' o encoding='latin1'
Columnas mal separadas	Delimitador incorrecto	Usa sep=';' en países con , como decimal
Decimales truncados o mal alineados	Falta float_format	Usa float_format='%.2f'

Ejercicio - Decisiones basadas en datos



Ejercicio

Decisiones basadas en datos

Objetivo: Aplicar técnicas de lectura, transformación y exportación de datos desde archivos CSV utilizando Python, simulando un análisis inicial de ventas para una empresa de distribución nacional.

Situación Eres parte del equipo de análisis comercial de una empresa que distribuye productos de consumo masivo a través de sucursales en todo Chile. Tu jefatura te ha enviado un archivo en formato CSV con el detalle de las ventas del último mes. Se espera que analices los datos y respondas:

- ¿Qué sucursales tienen un volumen de ventas mayor a lo esperado?
- ¿Qué productos están destacando en cada ciudad?
- ¿Cómo puedes preparar esta información para un informe que se enviará al equipo de gerencia?

Pero hay una condición: la gerencia exige trabajar con archivos .csv por su compatibilidad con su sistema de reportes, y espera que el proceso esté automatizado para repetirse mensualmente.



Ejercicio

Materiales necesarios



Archivo CSV

ventas_mensuales.csv



Entorno de desarrollo

Con Python y pandas instalados (VSCode, Jupyter, Google Colab).

Ejercicio

Instrucciones

Carga el archivo CSV proporcionado por la jefatura:

```
import pandas as pd
df = pd.read_csv('ventas_mensuales.csv')
```

Visualiza y comprende los datos:

```
print(df.head())
print(df.info())
```

¿Los nombres de las columnas tienen sentido?, ¿hay alguna columna innecesaria?, ¿algún dato mal escrito?

{desafío}
latam_

Ejercicio

Cálculo y análisis

Calcula una nueva columna llamada total_venta:

```
df['total_venta'] = df['cantidad'] * df['precio']
```

Identifica las ventas destacadas (mayores a \$10.000):

```
df_destacadas = df[df['total_venta'] > 10000]
```

¿Qué productos aparecen aquí? ¿Son productos de alto precio o de alta cantidad?

Ejercicio

Agrupación y exportación

Agrupar por sucursal y calcular el total acumulado:

```
ventas_por_sucursal = df.groupby('sucursal')['total_venta'].sum()  
print(ventas_por_sucursal)
```

Guardar el resultado en un nuevo archivo llamado ventas_resumen.csv, incluyendo solo sucursal y total de ventas:

```
ventas_por_sucursal.to_csv('ventas_resumen.csv', sep=';', header=True)
```

Validar que el archivo haya sido generado correctamente:

```
verificado = pd.read_csv('ventas_resumen.csv', sep=';')  
print(verificado)
```

{desafío}
latam_

Ejercicio

Preguntas de reflexión



Toma de decisiones

¿Qué decisiones podría tomar la gerencia a partir del archivo que generaste?



Limitaciones

¿Qué limitaciones ves en este análisis si solo usamos CSV y no bases de datos?



Automatización

¿Cómo podrías automatizar este proceso para hacerlo cada mes?

Resumen



Estructura CSV

Aprendimos qué es un archivo CSV, cómo está estructurado y por qué es tan utilizado.



Lectura

Practicamos la lectura con `pandas.read_csv()` considerando separadores, codificaciones y cabeceras.



Exportación

Exportamos DataFrames usando `to_csv()` con distintos parámetros.



Solución de problemas

Identificamos errores comunes y cómo resolverlos.



Próxima sesión...

En la siguiente sesión exploraremos el trabajo con archivos Excel en Python, considerando múltiples hojas, formatos y compatibilidades. Aprenderemos a leer y escribir .xlsx para ampliar nuestras fuentes de datos.